Binary Search Tree

A Friendly Introduction for First-Timers

Lab Handout — Data Structures

First, What is a Tree?

Nodes

Each box/circle that holds a value is called a node.

Root

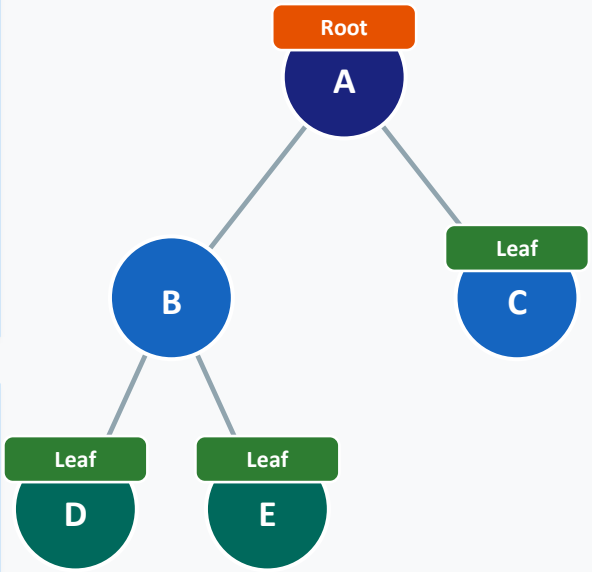
The node at the very top with no parent.

Edges

Lines connecting nodes are called edges.

Leaf

A node with no children — the end of a branch.

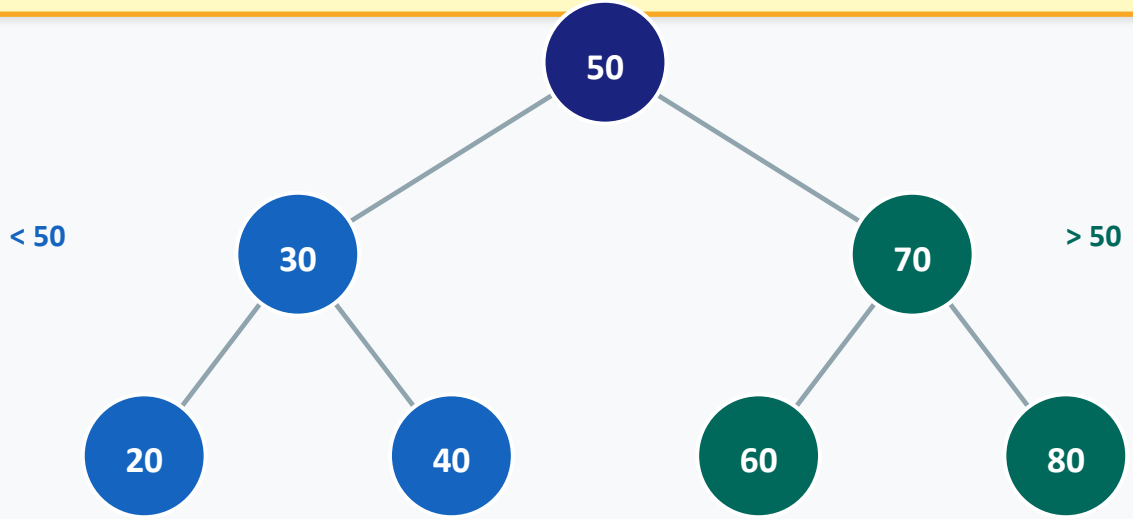


A tree diagram

The ONE Golden Rule of a BST

For every node:

LEFT child holds a value **SMALLER** | **RIGHT child** holds a value **LARGER**



Inserting a Value — Step by Step

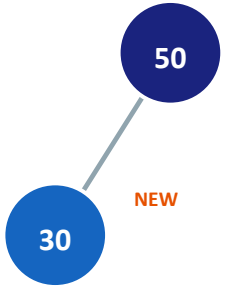
Let's insert: 50, 30, 70, 20 — one by one, following the BST rule

1 Insert 50

First node — becomes the Root

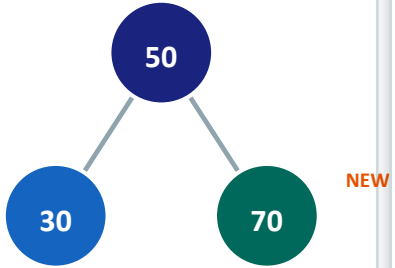
2 Insert 30

30 < 50 → go LEFT



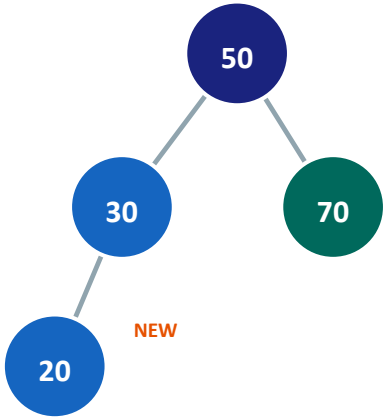
3 Insert 70

70 > 50 → go RIGHT



4 Insert 20

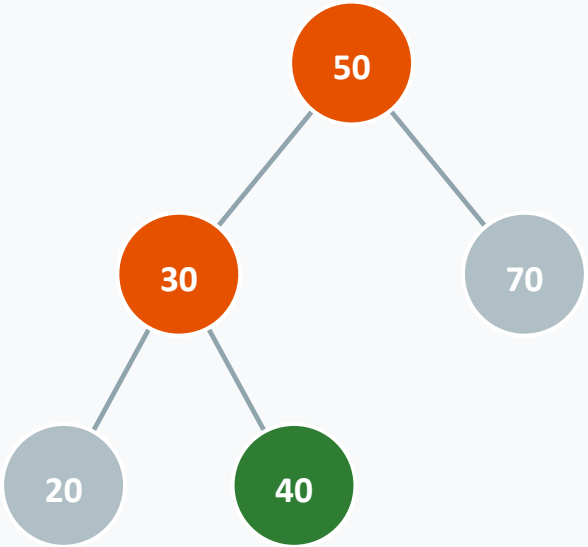
20<50→LEFT, 20<30→LEFT



Key idea: Always compare with the current node — go LEFT if smaller, RIGHT if larger. Stop at the first empty spot!

Searching — Like a Number Guessing Game!

Example: Find 40 in this tree



- Visited / Checked
- Found!
- Not visited

Search Path:

1. Node 50: 40 < 50 → go LEFT
2. Node 30: 40 > 30 → go RIGHT
3. **Node 40: 40 == 40 → FOUND!**

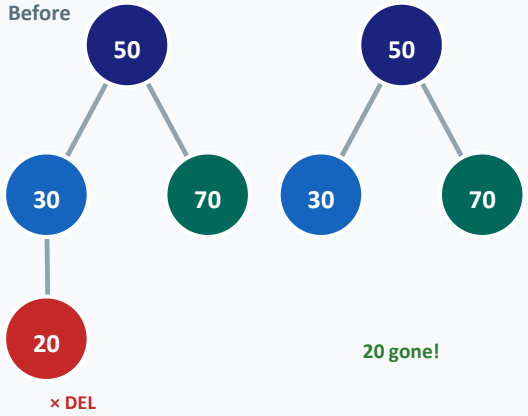
At most, we only check the HEIGHT of the tree!

Deleting a Node — 3 Cases

Deletion is trickier than insertion — depends on how many children the node has.

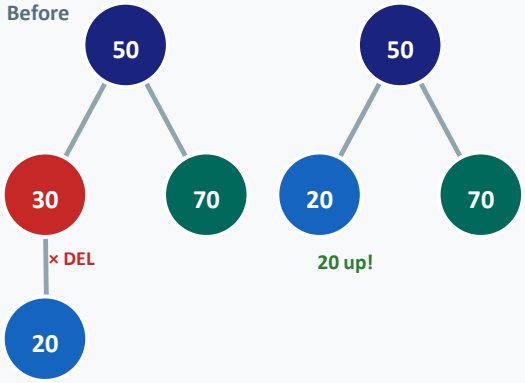
Case 1: No Children (Leaf)

Just remove the node. Nothing else to fix.



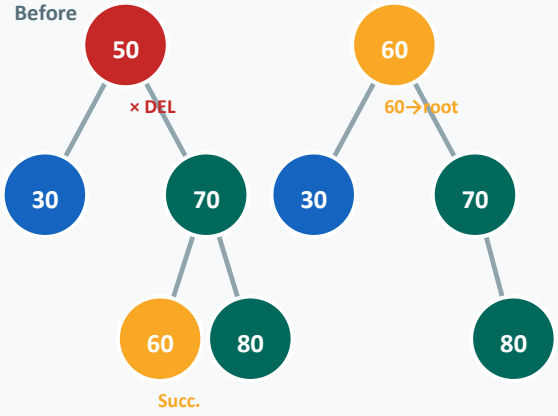
Case 2: One Child

Remove the node, promote its only child up.



Case 3: Two Children

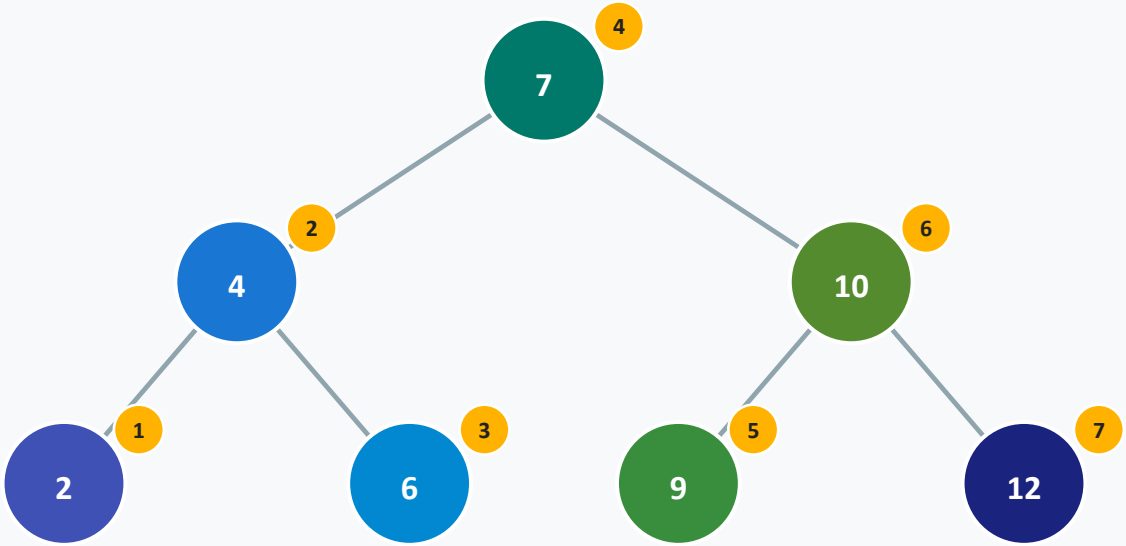
Replace with In-Order Successor (smallest in right subtree).



Case 3 tip: In-Order Successor = LEFT-MOST node of the RIGHT subtree. It's always the next-largest value!

In-Order Traversal = Sorted Output!

Reading a BST in-order (Left → Root → Right) always gives values sorted from smallest to largest.



Output: 2 → 4 → 6 → 7 → 9 → 10 → 12

Recipe: In-Order = visit Left subtree → then Root → then Right subtree — and repeat recursively!

Quick Recap

The Rule

Left = smaller, Right = larger.

Insertion

Compare & navigate until empty spot.

Search

Compare at each node, go left or right.

Delete

3 cases: leaf, one child, two children.

In-Order

Gives you all values sorted!

Why BSTs?

O(log n) avg. — fast search & insert.

Good luck in your lab! You've got this!